

Problem Set 1

Tate Mason

March 8, 2026

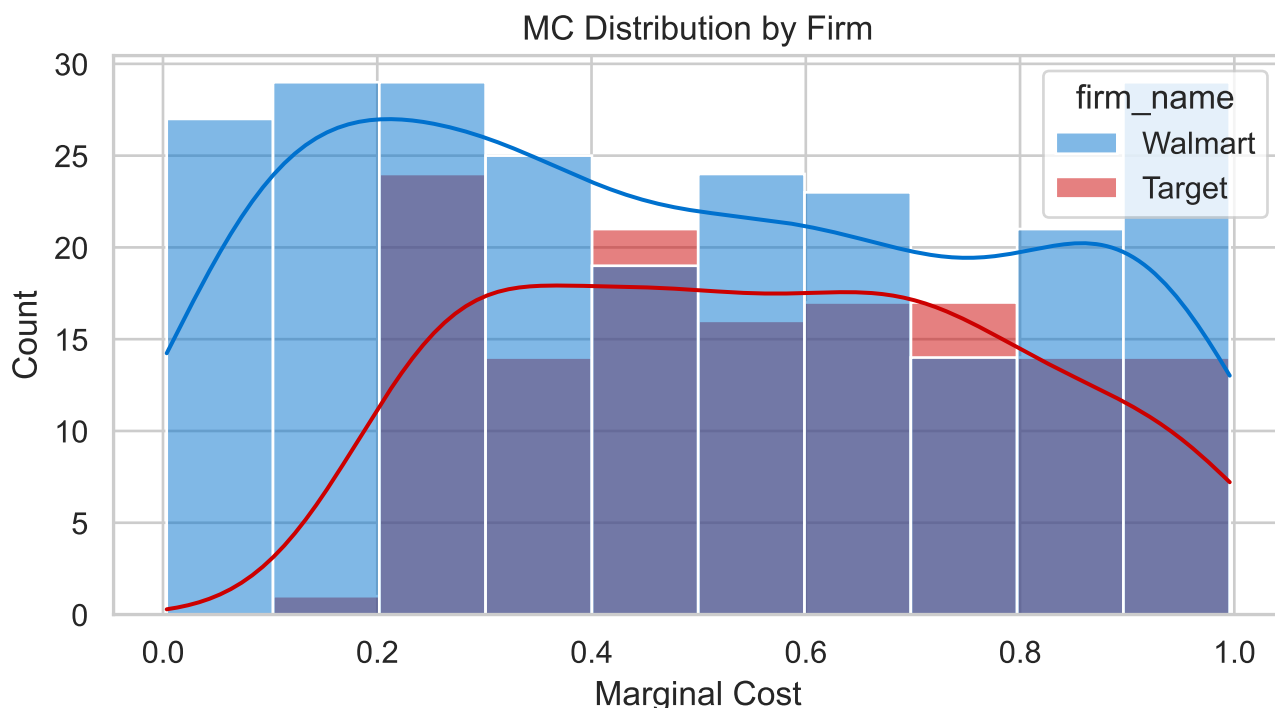
1 Introduction

Through this problem set, we were able to gain practical experience in differences between pricing regimes, and the efficacy of zone pricing. Following the work of Adams and Williams (2019) and Dellavigna and Gentzkow (2018), we will use simulated data on Walmart and Target. In the first problem, data facts will be presented, pointing out some interesting findings, as well as market level profits for both firms. Then, in the second, estimation results will be reported for OLS, 2SLS, and BLP methods. Finally, the final problem will take a closer look at the pricing regimes, presenting a best response table as well as consumer surplus under each permutation.

2 Data Facts

2.1 Income Sorting

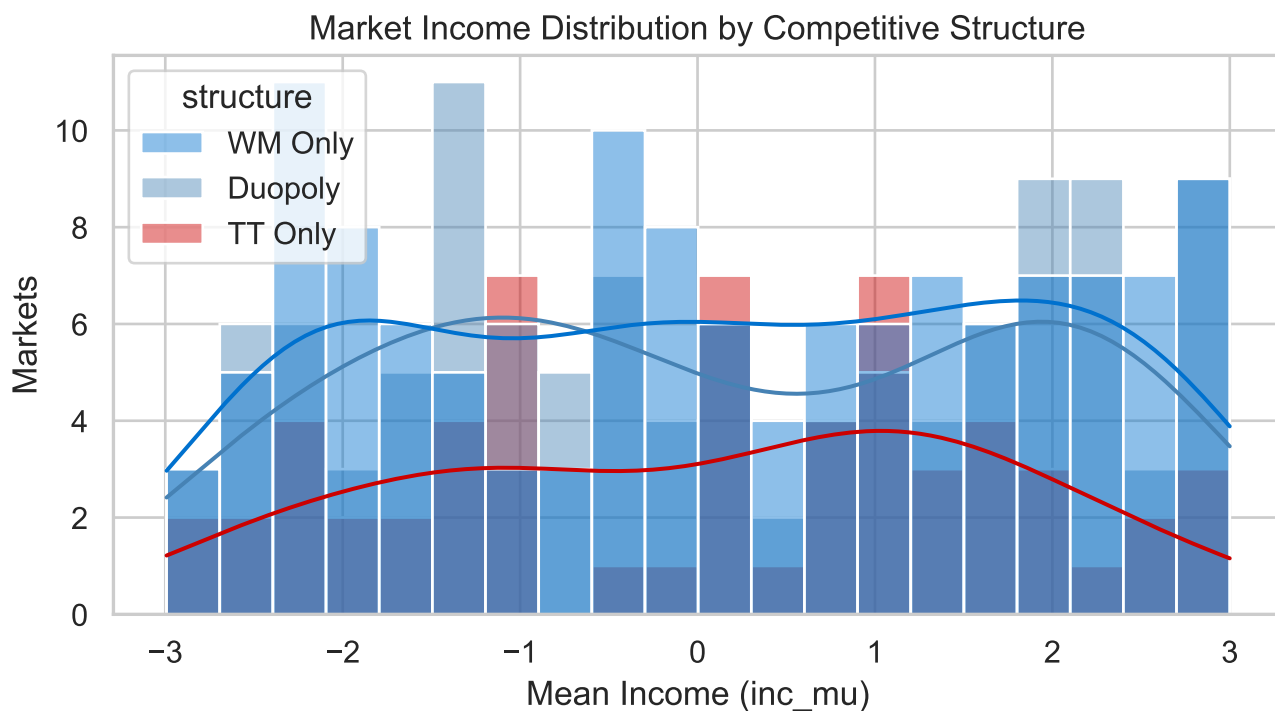
First, I would like to present a histogram of the distribution of market activity for each firm. For all plots, Walmart will be the blue firm and Target the red.



Walmart is able to run more cheaply, but with much more dispersion. Most of their firms are the low end of the marginal cost distribution, before tapering down. Target, however, is relatively stable between the 0.4-1.0 marginal cost buckets. Notice, however, Walmart is still more abundant across the entire cost distribution, save for two buckets.

2.2 Income Structure

Next I will present the distribution of income across the markets. This is important, as it will allow us to understand the heterogeneity in the consumer base, and how that may affect the pricing decisions of the firms.

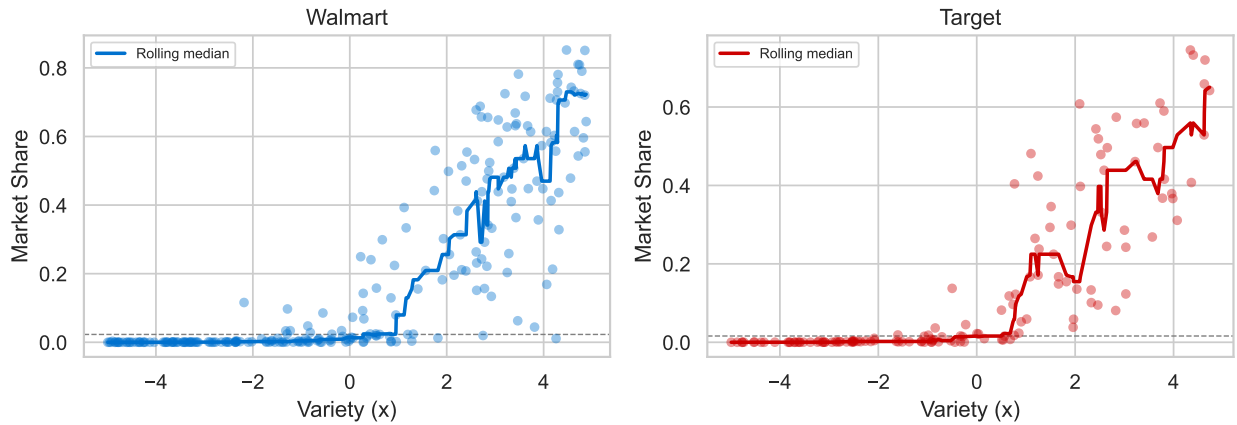


I will break this down in the three categories: only Walmart, only Target, and both. There are many more only Walmart markets across the entire mean income distribution. In fact, there are more only Walmart markets than markets with both firms as well, which is interesting. Target, on the other hand, is much less likely to be the only firm in a market, seeming to focus mostly on markets between -1 and 1 in mean income. The both category is closely linked with Walmart, a fact easily seen from the fit line. It would seem that income does not tell a ton.

2.3 Variety and Share

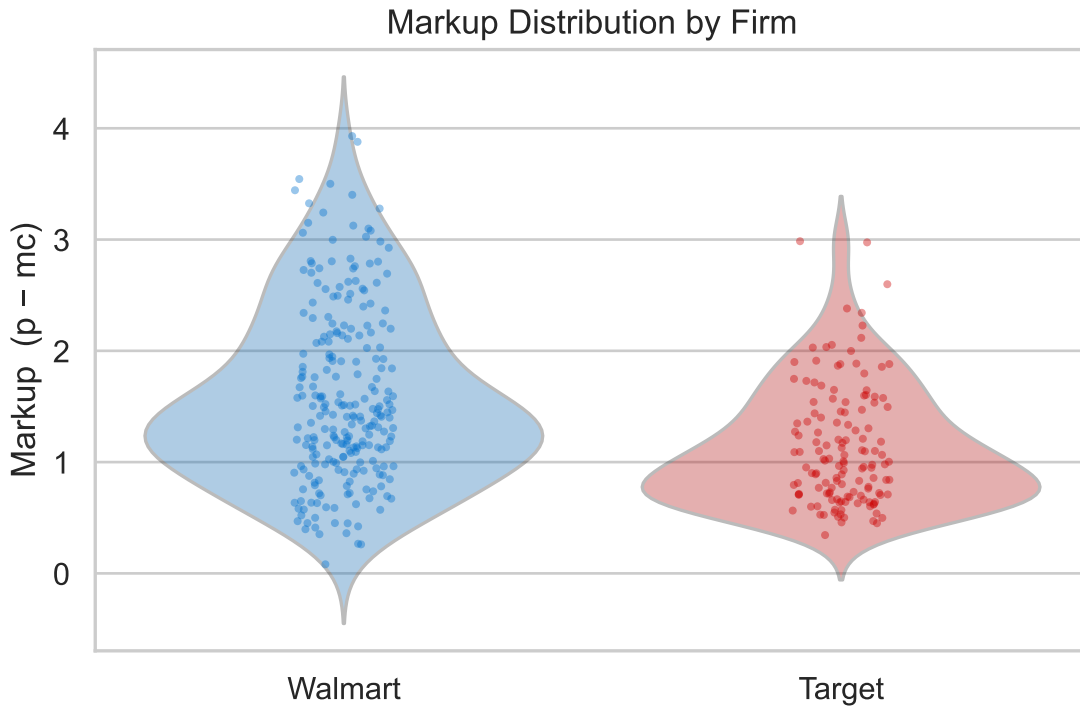
I was also curious if the variety in product characteristics had any relationship with the share of the market.

Variety and Market Share by Firm



Here, there is a positive relationship for both firms, but Walmart exhibits a much clearer trend, whereas Target has more noise. This indicates that Walmart is more likely to carry more varied products, while Target may specialize more.

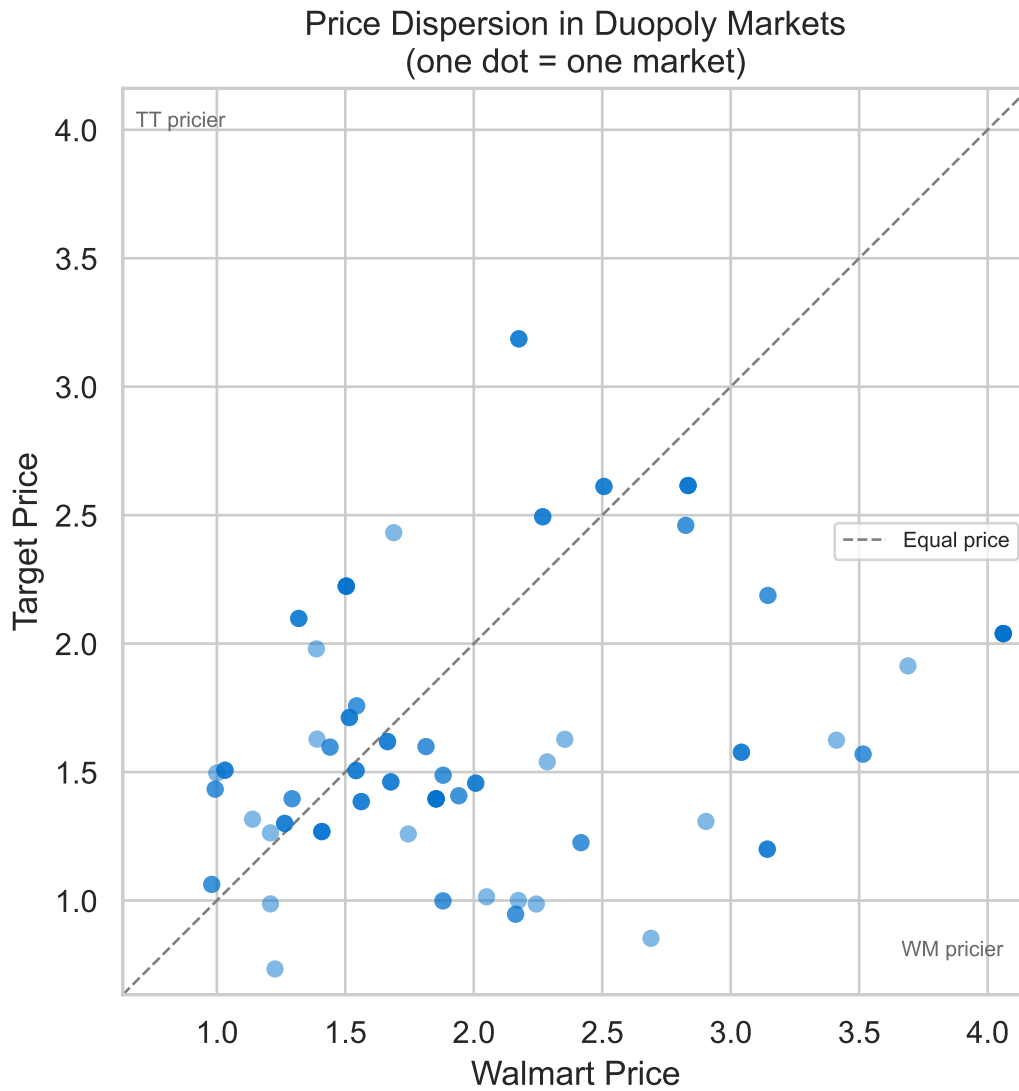
2.4 Markup Distribution



Markups are much more condensed for Target, with most of the markups between 0.5-2. Walmart, contrastingly, is much more dispersed, with mass spread between 0.25-4. This is interesting, as it points to Target having more control over their pricing and costs, while Walmart is more volatile. This could be due to the wide spread of markets that Walmart serves, leading to evidence of

monopoly power in some markets, and more competitive pressure in others.

2.5 Prices



This figure shows price competitions in duopoly markets. Walmart prices higher in a majority of the markets, and when Target does exert pricing power, they are closer to the competitive line than Walmart's deviations. This again points to Walmart having more market power, and in fact may point to zone pricing behavior.

3 Profits Table

Table 1: Profits by Market

	Walmart	Target
mean	324.28	185.0
median	31.79	14.26
std	499.72	318.98
min	0.0	0.0
max	2225.06	1486.5

Here, we can see that Walmart has higher profits as well as higher variance in profits. This is interesting, as it affirms the idea that Walmart is more likely to have market power in many markets, but it must compete in the markets where Target is present. Target's mean profits are almost half that of Walmart. The variance is also lower, but still very high given the mean. Even looking at the max profits, Walmart is able to max out at 150% that of Target, another indication that Walmart is much more able to exert market power.

A full csv of profits by markets is included in the data folder, but I shortened to the summery for the sake of presentation.

3.1 Data Facts Conclusion

It would appear from these figures that Walmart is more likely to have the market power advantage, either through monopoly or in competitive markets. Target, on the other hand, seems to be more of a niche player, staying in its lane with pricing and markups. These facts are interesting due to Target's reputation as a more upscale retailer. However, it would seem that Walmart is able to leeverage its size and scale to compete on its own terms, and thus is able to exert more market power in a majority of the provided markets.

4 Model and Estimations

In this section, I will present estimation results and brief interpretations of said reults. We will begin by reporting OLS results, before moving into 2SLS, and finally a BLP GMM procedure.

4.1 OLS

We first undertake a simple OLS, regressing the log difference between own and outside share on characteristics, price, and a constant.

	Parameter	Std. Error
Const.	-2.42	0.21
Characteristics	1.26	0.03
Price	-0.91	0.10

Table 2: OLS Estimates

	Parameter	Std. Error
Const.	-0.48	2.935
Characteristics	1.26	0.03
Price	-1.92	1.52

Table 3: 2SLS Estimates

$$\log(s_j - s_0) = \beta_0 + \beta_j x_j + \alpha_j p_j$$

such that

$$s_0 = 1 - \sum_{k \neq j} s_k$$

Going off of the true parameters, we can see the OLS estimates are biased away from zero. This is to be expected, as price is an endogenous variable, thus OLS is not going to produce a consistent estimate.

4.2 2SLS

Next, I will present the 2SLS estimates, using the marginal cost as an instrument for price.

$$\log(s_j - s_0) = \beta_0 + \beta_j x_j + \alpha_j p_j$$

$$p_j = \pi_0 + \pi_j x_j + \gamma_j mc_j$$

These estimates bias still bias α away from zero, but less than OLS. This is to be expected, since the MC instrument is weak, but still picks up some the variation. The standard error is also much higher, which is also expected, as the instrument is weak. The characteristics parameter is unaffected, as it is exogenous.

4.3 BLP

Finally, I will present the BLP GMM estimates, which should be the most consistent of the three. I will continue to instrument for price as in 2SLS, but will also include the interaction of characteristics

	Parameter	Std. Error
Const.	-1.37	2.51
Characteristics	1.21	0.06
Price	-1.80	1.36
Char. x Income	0.07	0.00
Income x MC	0.32	0.00

Table 4: BLP Estimates

and income, as well as income and MC. This will allow me to parse out the heterogeneity due to income.

My estimates are for the most part close to the true parameters set during the DGP. Only α_i is still pretty far, but the instruments are still weak, so I am not surprised.

5 Regime Comparison

This section will discuss the different pricing regimes, and what the best response is for each firm. First, however, I will briefly derive the FOC's for the firms, which will then be used in my functions to calculate the profits under each regime.

5.1 FOC's

Uniform Pricing: Under uniform pricing, the firms set a single price for all consumers.

$$\pi_j = \sum_t 1000 * (p_j - c_j) \cdot s_j$$

$$\frac{\partial \pi_j}{\partial p_j} = s_j + (p_j - c_j) \cdot \frac{\partial s_j}{\partial p_j} = 0$$

which implies

$$p_j = c_j - \frac{s_j}{\frac{\partial s_j}{\partial p_j}}$$

Zone Pricing: Under zone pricing, the firms can set prices which span zones depending on the market power of the firm in the two zones.

	Uniform	Zone	Market
Uniform	(49955, 15402)	(42160, 15911)	(39247, 16151)
Zone	(50809, 11500)	(42709, 11935)	(39755, 12178)
Market	(51269, 9796)	(42871, 10114)	(39874, 10395)

Table 5: Best Response Table

$$\begin{aligned}
\pi_{jz} &= \sum_{t \in Z} 1000 * (p_{jz} - c_{jt}) \cdot s_{jt} \\
\frac{\partial \pi_j}{\partial p_{jz}} &= \sum_{t \in Z} s_{jt} + (p_{jz} - c_{jt}) \cdot \frac{\partial s_{jt}}{\partial p_{jz}} = 0 \\
&\text{which implies} \\
p_{jz} &= \frac{\sum_{t \in Z} \frac{\partial s_{jt}}{\partial p_{jz}} \cdot c_{jt} - s_{jt}}{\sum_{t \in Z} \frac{\partial s_{jt}}{\partial p_{jz}}}
\end{aligned}$$

Market Pricing: Under market pricing, the firms can set prices for each market, thus maximizing profits in each market.

$$\begin{aligned}
\pi_{jt} &= 1000 * (p_{jt} - c_{jt}) \cdot s_{jt} \\
\frac{\partial \pi_{jt}}{\partial p_{jt}} &= s_{jt} + (p_{jt} - c_{jt}) \cdot \frac{\partial s_{jt}}{\partial p_{jt}} = 0 \\
&\text{which implies} \\
p_{jt} &= \frac{\frac{\partial s_{jt}}{\partial p_{jt}} \cdot s_{jt} - c_{jt}}{\frac{\partial s_{jt}}{\partial p_{jt}}}
\end{aligned}$$

Note: $\frac{\partial s_{jt}}{\partial p_{jz}} = \alpha s_{jt}(1 - s_{jt})$ with only notation differences between the regimes.

5.2 Best Response Table

From the table, I will concede I have made a mistake. Walmart should not want to choose unifrom, as it can exert any pricing regime due to Target's lack of market power. So, Walmart should want to choose market pricing and target should want to choose uniform. However, I have that Walmart chooses uniform and Target chooses zone. I will have to go back to see what the error is stemming from, but I know this is not the correct equilibrium.

If you see any glaring errors in my code, attached at the end of the writeup, please let me know.

6 Conclusion

In conclusion, I have presented some data facts, estimation results, and a regime comparison for the problem set. I will have to go back and check my code for the regime comparison, as the results are not what I expected. However, I still feel that this exercise was very useful in a few dimensions. Firstly, it was a good refresher on the different estimation techniques, and the biases that can result from misspecified methods of computation. Secondly, it was good for thinking of pricing games and how to derive the FOC's for the different regimes. Finally, the data exploration exercise was good as I am trying to learn Python and its plotting libraries.

A Code

```
import numpy as np
import pandas as pd
import scipy as sp
import matplotlib.pyplot as plt
from numpy import std
import statsmodels.api as sm
from statsmodels.sandbox.regression.gmm import IV2SLS
from scipy.optimize import root
import seaborn as sns

# setting seaborn settings and color scheme
sns.set_theme(style="whitegrid", font_scale=1.05)
WM_COLOR = "#0071CE" # Walmart blue
TT_COLOR = "#CC0000" # Target red

# defining the palette to call in seaborn
palette = {"Walmart": WM_COLOR, "Target": TT_COLOR}

"""
Main_file_for_Problem_Set_1_-_IO2
"""

# loading data
df = pd.read_csv('../Data/full_data_zone.csv')
df_l = pd.read_csv('../Data/full_data_zone_long.csv')

# calculating profits and markups
df_l["profit"] = 1000*df_l["share"]*(df_l["p"] - df_l["mc"])
df_l["markup"] = df_l["p"] - df_l["mc"]

active = df_l[df_l["active"]==1] # subsetting to active markets

active["firm_name"] = active["firm"].map({1:"Walmart", 2:"Target"}) # label for graphs

# --- MC dist Across Mkt --- #

fig, ax = plt.subplots(figsize=(7,4))

"""
Learning_seaborn_pardon_the_nuisance:

data_confirms_dataset_used
x_determines_variable_of_interest
hue_splits_data_by_group_and_then_colors_the_same
kde_fits_a_smooth_density_curve
alpha_controls_transparency_allowing_forBars_to_overlap
```

```

"""

# creating histogram of MC dist by firm
sns.histplot(data=active, x="mc", hue="firm_name",
              palette=palette, kde=True, alpha=0.5, ax=ax)
ax.set_title("MC_Distribution_by_Firm")
ax.set_xlabel("Marginal_Cost")
ax.set_ylabel("Count")
plt.tight_layout()
plt.savefig("../Graphics/mc_dist.pdf") # saving figure to graphics folder

# --- Variety vs Share --- #
# split panel scatter + rolling median; strong 0.8 corr per firm

# creating subset of data for duopoly markets
both = df[(df["active1"]==1) & (df["active2"]==1)].copy()

active = active.copy()

fig, axes = plt.subplots(1, 2, figsize=(10, 4), sharey=False)

# looping through firms and plotting scatter + rolling median for each
for ax, (firm, color) in zip(axes, [("Walmart", WM_COLOR), ("Target", TT_COLOR)]):
    sub = active[active["firm_name"] == firm].sort_values("x")
    ax.scatter(sub["x"], sub["share"], color=color, alpha=0.4, s=20, zorder=2)
    rolling_med = sub["share"].rolling(window=15, center=True, min_periods=5).median()
    ax.plot(sub["x"], rolling_med, color=color, lw=2, zorder=3, label="Rolling_median")
    )
    ax.axhline(sub["share"].median(), color="grey", lw=0.8, ls="--", zorder=1)
    ax.set_title(firm)
    ax.set_xlabel("Variety_(x)")
    ax.set_ylabel("Market_Share")
    ax.legend(fontsize=8)

fig.suptitle("Variety_and_Market_Share_by_Firm", y=1.01)
plt.tight_layout()
plt.savefig("../Graphics/variety_share.pdf", bbox_inches="tight") # saving figure to
graphics folder

# --- Price Dispersion Across Markets --- #
# WM vs TT price per duopoly market; 45-degree line = equal price

fig, ax = plt.subplots(figsize=(6, 6))

# creating scatter of WM vs TT price in duopoly markets
ax.scatter(both["p1"], both["p2"], color=WM_COLOR, alpha=0.5, s=30, zorder=2)

```

```

lim_lo = min(both["p1"].min(), both["p2"].min()) - 0.1
lim_hi = max(both["p1"].max(), both["p2"].max()) + 0.1
ax.plot([lim_lo, lim_hi], [lim_lo, lim_hi], color="grey", lw=1, ls="--", zorder=1,
        label="Equal_price")
ax.text(lim_hi - 0.1, lim_lo + 0.15, "WM_pricier", ha="right", fontsize=8, color="
        dimgrey")
ax.text(lim_lo + 0.05, lim_hi - 0.15, "TT_pricier", ha="left", fontsize=8, color="
        dimgrey")
ax.set_xlim(lim_lo, lim_hi)
ax.set_ylim(lim_lo, lim_hi)
ax.set_aspect("equal")
ax.set_xlabel("Walmart_Price")
ax.set_ylabel("Target_Price")
ax.set_title("Price_Dispersion_in_Duopoly_Markets\n(one_dot=_one_market)")
ax.legend(fontsize=8)
plt.tight_layout()
plt.savefig("../Graphics/price_dispersion.pdf") # saving figure to graphics folder

# --- Income Distribution by Market Structure --- #
# inc_sig is constant; inc_mu is the meaningful variation
# shows whether richer/poorer markets sort into different structures

df["structure"] = np.where(
    (df["active1"]==1) & (df["active2"]==1), "Duopoly",
    np.where(df["active1"]==1, "WM_Only", "TT_Only")
)

mkt_inc = df[["market", "inc_mu", "structure"]].drop_duplicates()

struct_palette = {"Duopoly": "steelblue", "WM_Only": WM_COLOR, "TT_Only": TT_COLOR}
struct_order   = ["WM_Only", "Duopoly", "TT_Only"]

fig, ax = plt.subplots(figsize=(7, 4))
sns.histplot(data=mkt_inc, x="inc_mu", hue="structure",
             hue_order=struct_order, palette=struct_palette,
             kde=True, alpha=0.45, bins=20, ax=ax)
ax.set_title("Market_Income_Distribution_by_Competitive_Structure")
ax.set_xlabel("Mean_Income_(inc_mu)")
ax.set_ylabel("Markets")
plt.tight_layout()
plt.savefig("../Graphics/income_structure.pdf")

# --- Markup Distribution --- #
# violin + strip replaces plain boxplot      shows full shape

active["markup"] = active["p"] - active["mc"]

fig, ax = plt.subplots(figsize=(6, 4))

```

```

sns.violinplot(data=active, x="firm_name", y="markup", hue="firm_name",
               palette=palette, inner=None, alpha=0.35, legend=False,
               order=["Walmart", "Target"], ax=ax)
sns.stripplot(data=active, x="firm_name", y="markup", hue="firm_name",
              palette=palette, alpha=0.4, size=3, jitter=True, legend=False,
              order=["Walmart", "Target"], ax=ax)
ax.set_title("Markup_Distribution_by_Firm")
ax.set_xlabel("")
ax.set_ylabel("Markup_((p - mc))")
plt.tight_layout()
plt.savefig("../Graphics/markup_dist.pdf")

# ===== #
# (2.3) Profit in each market #
# ===== #

active["profit"] = 1000 * active["share"] * (active["p"] - active["mc"])

profits_table = active.pivot_table(
    index = "market",
    columns = "firm",
    values = "profit",
    aggfunc = "sum"
)

with pd.option_context(
    "display.max_rows", None,
    "display.max_columns", None,
    "display.width", None
):
    print(profits_table)

profits_table.columns = ["Walmart", "Target"]
profits_table = profits_table.round(2)
profits_table.to_csv("../Data/profits_table.csv")

summary = profits_table.agg(["mean", "median", "std", "min", "max"]).round(2)
summary.to_csv("../Data/profits_summary.csv")
# ===== #
# (3.1) MNL Estimation #
# ===== #

# ===== #
# (A) Estimation (Uninstrumented) #
# ===== #

```

```

active["outside_share"] = 1 - active.groupby("market")["share"].transform("sum")

active["DV"] = np.log(active["share"]) - np.log(active["outside_share"])

X_col = ["x", "p"]

active = active.replace([np.inf, -np.inf], np.nan).dropna(subset=["DV", "x", "p", "mc"
    ])

X = sm.add_constant(active[X_col])
Y = active["DV"]

res = sm.OLS(Y, X).fit()

print(res.summary())

# ===== #
# (A) Estimation (Instrumented)      #
# ===== #

Z_col = ["x", "mc"]
Z = sm.add_constant(active[Z_col])

iv = IV2SLS(Y, X, Z).fit()
print(iv.summary())

# ===== #
# (3.2) MNL Estimation (True)       #
# ===== #

# ===== #
# (A) Estimation                    #
# ===== #

import pyblp

# base formulation definition
X1 = pyblp.Formulation('1_+_x_+_prices')
X2 = pyblp.Formulation('0_+_x_+_prices')

agent_form = pyblp.Formulation('0_+_income')

# product data definition

# adhering to PyBLP naming
prod_data = active.rename(columns={"market": "market_ids", "share": "shares", "p": "
    prices"}).copy()
# defining the product data

```

```

prod_data = prod_data[["market_ids", "shares", "prices", "x", "mc", "firm", "zone"]]

# market level inc_mu, inc_sigma
mkt_demo = (
    active.groupby("market")[["inc_mu", "inc_sig"]]
    .mean()
    .reset_index()
)

# merging product data and market demos
prod_data = prod_data.merge(mkt_demo, left_on="market_ids", right_on="market", how = "
    left")

# defining what our instrument is for price
prod_data["demand_instruments0"] = prod_data["mc"]
prod_data["demand_instruments1"] = prod_data["x"] * prod_data["inc_mu"]
prod_data["demand_instruments2"] = prod_data["mc"] * prod_data["inc_mu"]

# agent data

# seed for reproducing
rng = np.random.default_rng(219)

# n
R = 500

# mean and variance of income
m = np.maximum(mkt_demo["inc_mu"].to_numpy(dtype=float), 1e-12)
s = np.maximum(mkt_demo["inc_sig"].to_numpy(dtype=float), 0.0)

sig2 = np.log1p((s**2) / (m**2))
mu_ln = np.log(m) - 0.5 * sig2
sig_ln = np.sqrt(sig2)

# defining markets
markets = mkt_demo["market"].to_numpy()
# market vector
M = len(markets)

# drawing income from mu, sigma size (M,R)
income_draws = rng.lognormal(mean = mu_ln[:, None], sigma = sig_ln[:, None], size = (M
    ,R))

# defining agent data
agent_data = pd.DataFrame({
    "market_ids": np.repeat(markets, R), # size of market ids
    "weights": np.full(M*R, 1.0/R), # weights size (M*R, 1/R)
    "income": income_draws.ravel() # flattens our income draws
})

```



```

# defining what PyBLP is solving
problem = pyblp.Problem(
    product_formulations = (X1, X2),
    product_data = prod_data,
    agent_formulation = agent_form,
    agent_data = agent_data,
    integration = pyblp.Integration("mlhs", size=R)
)

# sigma matrix 2x2 of 0's
sigma0 = np.diag([0,0])
# pi matrix 2x1 of 0's
pi0 = np.array([[0.001], [0.001]])

b0 = np.array([1,1,1])
# calling solution to problem
optimization = pyblp.Optimization("bfgs", {'gtol':1e-10, 'maxiter':1000})
results = problem.solve(beta=b0,sigma=sigma0, pi = pi0, method="2s", optimization=
    optimization)
print(results.pi)
print(results.sigma)

# ===== #
# Counterfactual Analysis      #
# ===== #

# extracting estimated betas from last section
beta = results.beta.flatten()
beta_0 = beta[0]
beta_x = beta[1]
beta_p = beta[2] # alpha -- should be negative

def compute_shares(df, prices):
    df = df.copy()
    df["prices"] = prices

    shares = np.zeros(len(df))

    for m, sub in df.groupby("market_ids"):
        idx = sub.index
        delta = beta_0 + beta_x * sub["x"].values + beta_p * prices[idx]

        exp_delta = np.exp(delta)
        denom = 1 + exp_delta.sum()

        shares[idx] = exp_delta / denom

```

```

    return shares

def compute_profit(df, prices):
    shares = compute_shares(df, prices)

    df = df.copy()
    df["shares_new"] = shares
    df["profit"] = 1000 * df["shares_new"] * (prices - df["mc"])

    return df

# ----- #
# Build price vectors                                #
# ----- #

def uniform_prices(p, df, firm, p_val):
    p = p.copy()
    p[df["firm"].values == firm] = p_val
    return p

def zone_prices(p, df, firm, p_vec):
    """p_vec: array of length = number of zones, one price per zone"""
    p = p.copy()
    for i, z in enumerate(np.sort(df["zone"].unique())):
        mask = (df["firm"].values == firm) & (df["zone"].values == z)
        p[mask] = p_vec[i]

    return p

def market_prices(p, df, firm, p_vec):
    """p_vec: array of length = number of markets, one price per market"""
    p = p.copy()

    for i, m in enumerate(np.sort(df["market_ids"].unique())):
        mask = (df["firm"].values == firm) & (df["market_ids"].values == m)
        p[mask] = p_vec[i]

    return p

# ----- #
# FOC functions                                      #
# ----- #

def uniform_foc(p_val, df, firm):

```

```

p_val = p_val[0]
p_base = df["prices"].values.copy()
p = uniform_prices(p_base, df, firm, p_val)
s = compute_shares(df, p)

own = df["firm"].values == firm
mc = df["mc"].values

foc = np.sum(
    s[own] + (p_val - mc[own]) * beta_p * s[own] * (1 - s[own])
)
return [foc]

def zone_foc(p_vec, df, firm):
    p_vec = np.asarray(p_vec)
    p_base = df["prices"].values.copy()
    p = zone_prices(p_base, df, firm, p_vec)
    s = compute_shares(df, p)

    zones = np.sort(df["zone"].unique())
    focs = np.zeros(len(zones))
    mc = df["mc"].values

    for i, z in enumerate(zones):
        mask = (df["firm"].values == firm) & (df["zone"].values == z)
        focs[i] = np.sum(
            s[mask] + (p_vec[i] - mc[mask]) * beta_p * s[mask] * (1 - s[mask])
        )

    return focs

def market_foc(p_vec, df, firm):
    p_vec = np.asarray(p_vec)
    p_base = df["prices"].values.copy()
    p = market_prices(p_base, df, firm, p_vec)
    s = compute_shares(df, p)

    markets = np.sort(df["market_ids"].unique())
    focs = np.zeros(len(markets))
    mc = df["mc"].values

    for i, m in enumerate(markets):
        mask = (df["firm"].values == firm) & (df["market_ids"].values == m)
        focs[i] = np.sum(
            s[mask] + (p_vec[i] - mc[mask]) * beta_p * s[mask] * (1 - s[mask])
        )

```

```

    return focs

# ----- #
# Solve FOCs #
# ----- #

zones = np.sort(prod_data["zone"].unique())
markets = np.sort(prod_data["market_ids"].unique())

# --- Uniform ---
uni_sol_f1 = root(lambda p: uniform_foc(p, prod_data, firm=1), x0=[2.0])
uni_sol_f2 = root(lambda p: uniform_foc(p, prod_data, firm=2), x0=[2.0])
# --- Zone ---
z_sol_f1 = root(lambda p: zone_foc(p, prod_data, firm=1), x0=np.full(len(zones), 2.0))
z_sol_f2 = root(lambda p: zone_foc(p, prod_data, firm=2), x0=np.full(len(zones), 2.0))
# --- Market ---
m_sol_f1 = root(lambda p: market_foc(p, prod_data, firm=1), x0=np.full(len(markets),
    2.0))
m_sol_f2 = root(lambda p: market_foc(p, prod_data, firm=2), x0=np.full(len(markets),
    2.0))

# ----- #
# Build price vectors for 9 regimes #
# ----- #

def get_price_vector(df, regime_f1, regime_f2):
    # start from current prices, then overwrite firm by firm
    p = df["prices"].values.copy()

    for firm, regime, sol_z, sol_m, uni_p in [
        (1, regime_f1, z_sol_f1, m_sol_f1, uni_sol_f1),
        (2, regime_f2, z_sol_f2, m_sol_f2, uni_sol_f2),
    ]:
        if regime == "uniform":
            p = uniform_prices(p, df, firm, uni_p.x[0])
        elif regime == "zone":
            p = zone_prices(p, df, firm, sol_z.x)
        elif regime == "market":
            p = market_prices(p, df, firm, sol_m.x)

    return p

# ----- #
# Profits & CS for all 9 regimes #

```

```

# ----- #

def compute_cs(df, prices):
    cs = 0.0
    df = df.copy()
    df["prices"] = prices
    for m, sub in df.groupby("market_ids"):
        idx = sub.index
        delta = beta_0 + beta_x * sub["x"].values + beta_p * prices[idx]
        cs += np.log(1 + np.exp(delta).sum()) / (-beta_p)
    return cs * 1000 # scale by market size

regime_list = ["uniform", "zone", "market"]
results_table = []

for r1 in regime_list:
    for r2 in regime_list:
        p = get_price_vector(prod_data, r1, r2)
        df_res = compute_profit(prod_data, p)
        pi1 = df_res[df_res["firm"] == 1]["profit"].sum()
        pi2 = df_res[df_res["firm"] == 2]["profit"].sum()
        cs = compute_cs(prod_data, p)

        results_table.append({
            "Firm_1": r1, "Firm_2": r2,
            "Profit_F1": round(pi1, 0),
            "Profit_F2": round(pi2, 0),
            "CS": round(cs, 0),
        })

results_df = pd.DataFrame(results_table)

print("\n==_Profits_and_CS_across_all_9_regimes_==")
print(results_df.to_string(index=False))

```